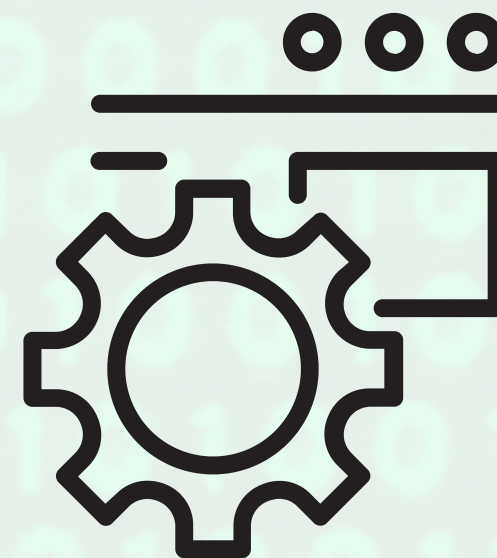




AI Image

Detector





ข้อมูลที่ใช้สอน (Dataset Overview)

แหล่งข้อมูล (Data Sources) รวม 4 Datasets ใหญ่

1. 140k Real and Fake Faces เป็นฐานข้อมูลหลักสำหรับโครงหน้าคน (AI รุ่น StyleGAN)
2. FFHQ (Flickr-Faces-HQ) ภาพคนจริงคุณภาพสูง ใช้แก้ปัญหา "คนใส่ฟิลเตอร์/แต่งรูป" เพื่อไม่ให้ AI มองว่าเป็นภาพปลอม
3. CIFAKE ภาพทั่วไป (สัตว์, ยานพาหนะ, สิ่งของ) เพื่อให้โมเดลแยกแยะวัตถุได้ด้วย (AI รุ่น Stable Diffusion 1.4)
4. GenAI (Midjourney & DALL-E) ภาพ AI ยุคใหม่ที่มีความละเอียดสูงและเนียนสมจริง เพื่ออัปเดตให้โมเดลทันโลก

การจัดการข้อมูล (Data Preparation)

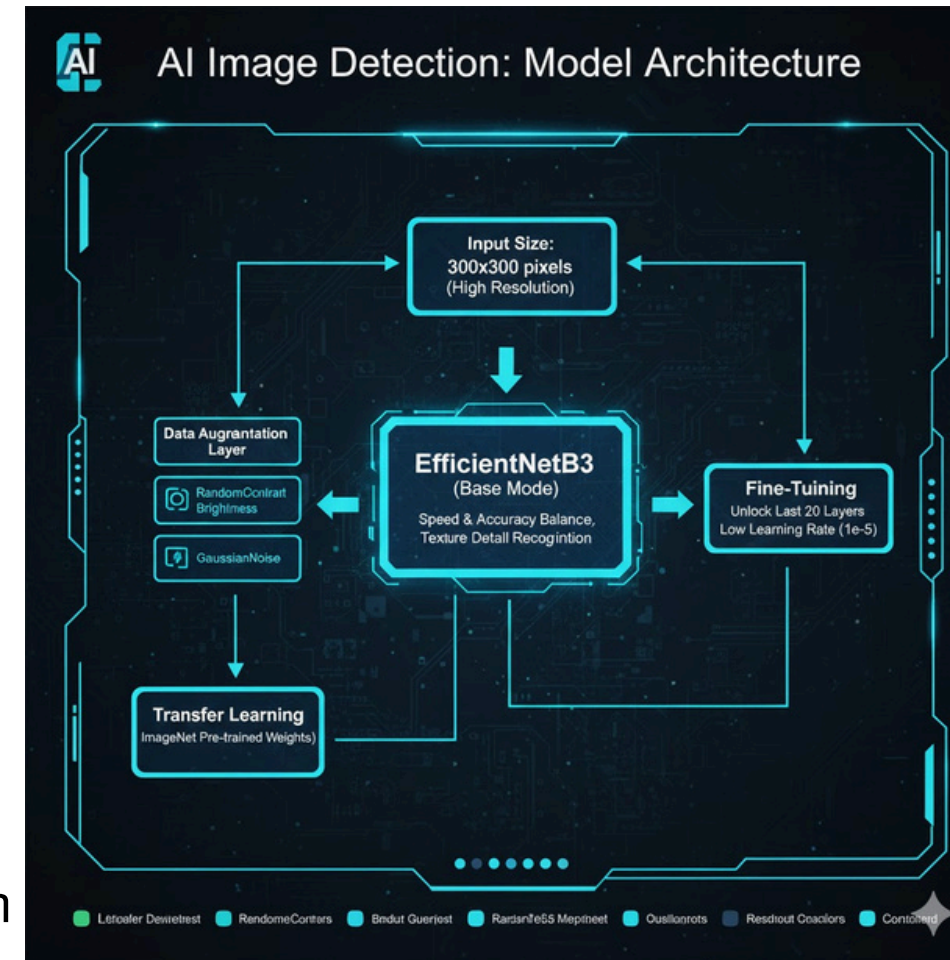
1. Balancing ปรับสมดุลข้อมูล Real และ Fake ให้ใกล้เคียงกัน (50:50) เพื่อไม่ให้โมเดลลำเอียง
2. Splitting แบ่งข้อมูล Train 80% และ Test 20% (โดยไม่ให้รูปซ้ำกัน)



สถาปัตยกรรมโมเดล (Model Architecture)

Base Model: EfficientNetB3

- เหตุผลที่เลือก: มีความสมดุลระหว่างความเร็วและความแม่นยำ เหมาะกับการจับรายละเอียด Texture เล็กๆ (เช่น รูขุมขน vs ผิวพลาสติก) ได้ดีกว่ารุ่น B0
- Input Size: 300x300 pixels (ความละเอียดสูงเพื่อให้เห็นร่องรอย AI)



เทคนิคพิเศษ (Key Techniques)

01

Data Augmentation Layer

ฝังกระบวนการแต่งภาพเข้าไปในโมเดล (RandomContrast, RandomBrightness, GaussianNoise) เพื่อสอนให้ AI คุ้นต่อภาพเบลอหรือภาพแต่งแสง

02

Transfer Learning

ใช้น้ำหนักเริ่มต้นจาก ImageNet เพื่อให้รู้จักรูปทรงพื้นฐานก่อน

03

Fine-Tuning

ปลดล็อก 20 เลเยอร์สุดท้าย และเทรนด้วย Learning Rate ต่ำ (1e-5e) เพื่อให้เรียนรู้ลักษณะเฉพาะของภาพ AI



AI Image Detector ✨

Upload an image to detect if it's Real or AI-Generated.



Reset

AI Artifact Scanner V2.0

Analysis Result

FAKE (ภาพจาก AI)

94.59% Confidence

```
{
  "label": "Fake (ภาพจาก AI)",
  "score": 0.05406312271952629,
  "confidence": 94.59368772804737
}
```

How to use

1. Click "Choose File" to select an image.
2. Click "Analyze Image" to start.
3. Wait for the result: Real or Fake.

เครื่องมือและไลบรารี (Tools & Libraries)

ภาษา (Language) Python 3.10+

Core Libraries

1. TensorFlow / Keras: เฟรมเวิร์กหลักในการสร้างและเทรน Deep Learning Model
2. NumPy & Pandas: จัดการข้อมูลและคำนวณทางคณิตศาสตร์
3. Scikit-learn: คำนวณค่าความแม่นยำ (Classification Report, Confusion Matrix)
4. OpenCV & Matplotlib: จัดการรูปภาพและแสดงผลกราฟ

Deployment

1. Render สร้าง Web Server (Backend API)

Hardware

1. GPU: NVIDIA GeForce RTX 3060 Ti (ใช้เทคนิค Mixed Precision float16 เพื่อเร่งความเร็ว)



ขั้นตอนการพัฒนา (Workflow)

01

Data Collection

รวบรวมและคัดแยกภาพเสีย
(Corrupted Images)

02

Preprocessing

แปลงขนาดภาพและปรับค่าสี
(Normalization)

Training Phase

Round

01

Train เฉพาะส่วนหัว (Head)

Round

02

Fine-Tuning ส่วนท้ายของ EfficientNetB3

Round

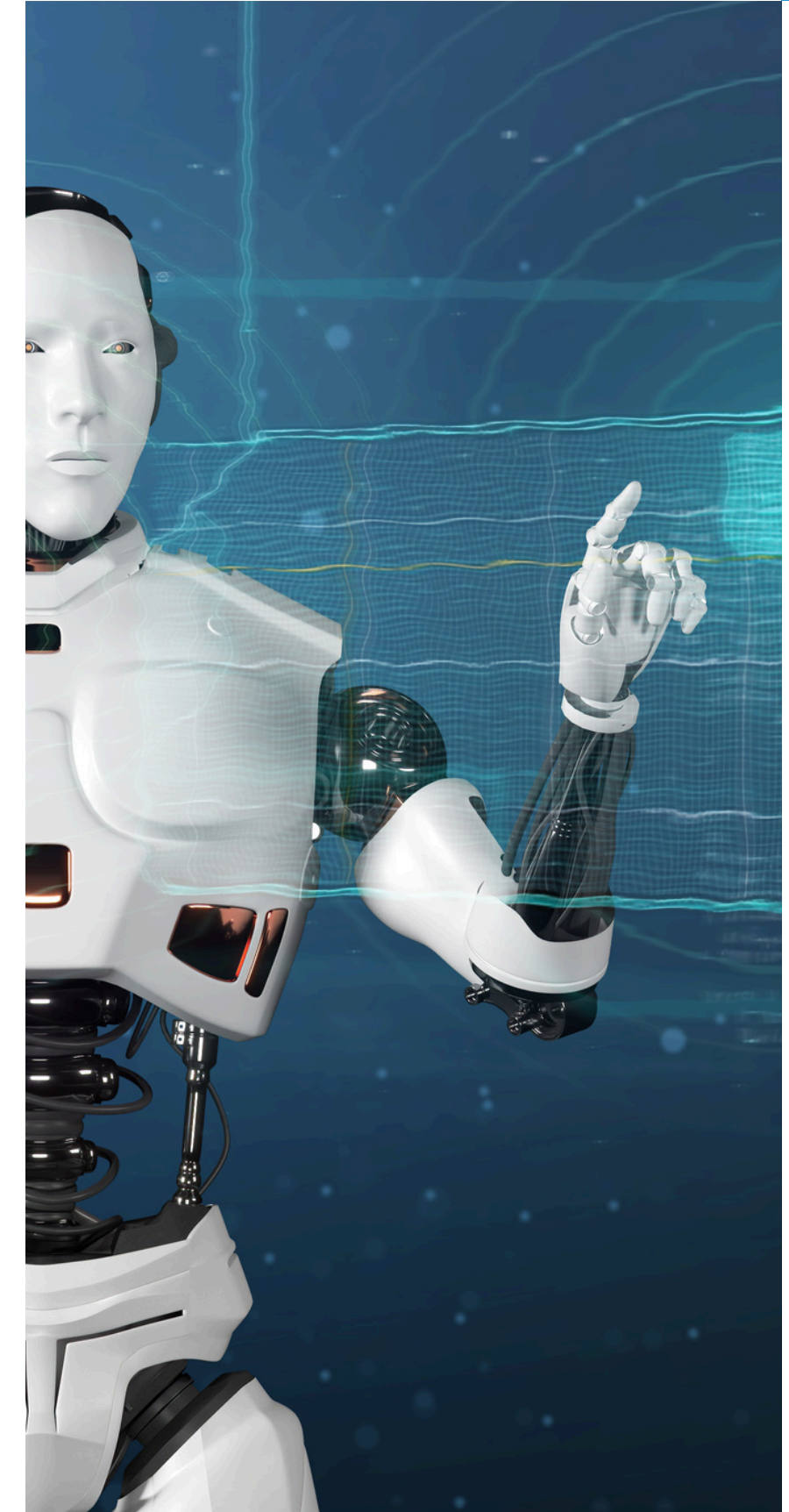
03

วัดผลด้วย Test Set ที่ไม่เคยเห็นมาก่อน

Round

04

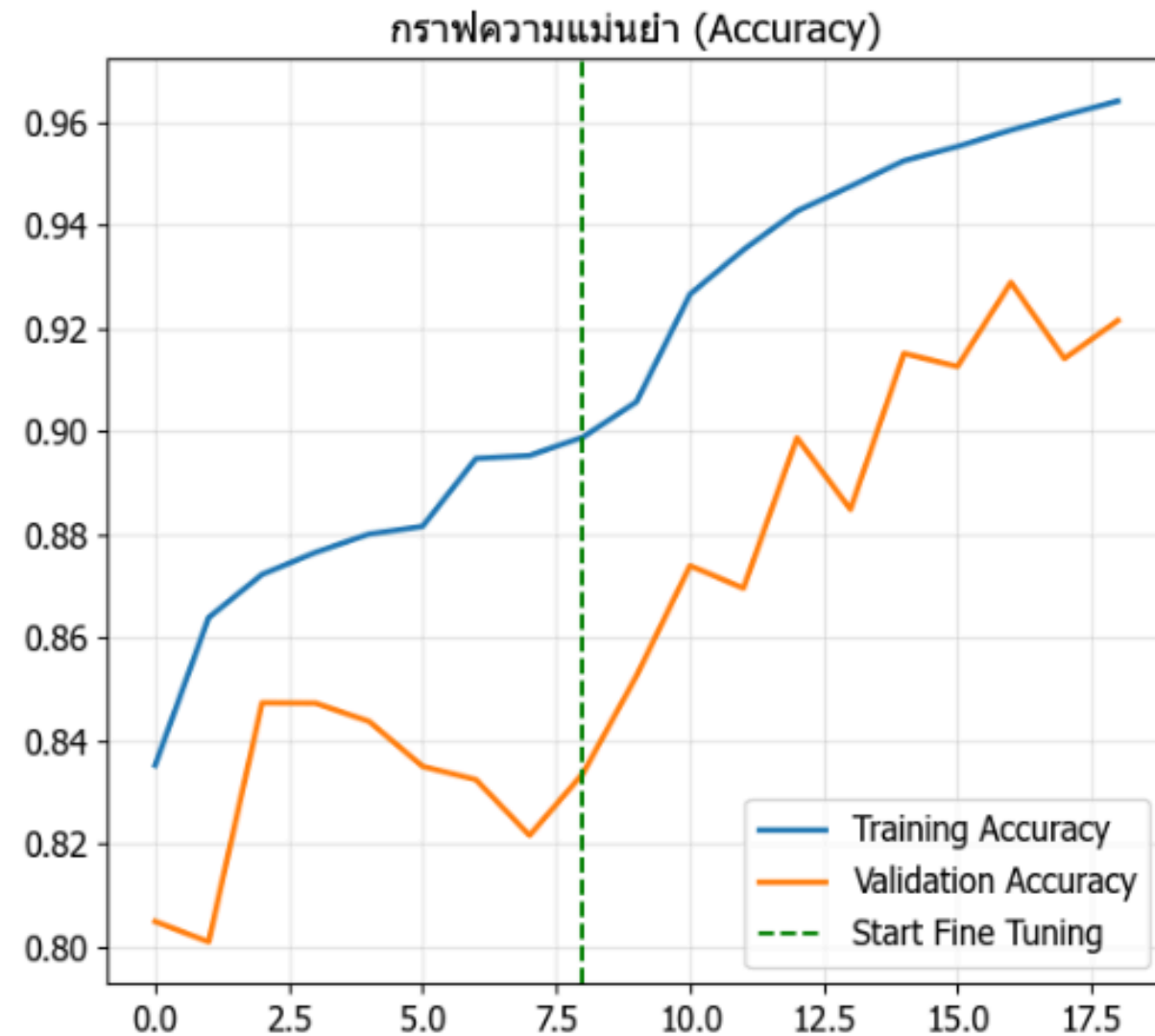
บันทึกโมเดล (.h5) และสร้างเว็บสำหรับใช้งานจริง



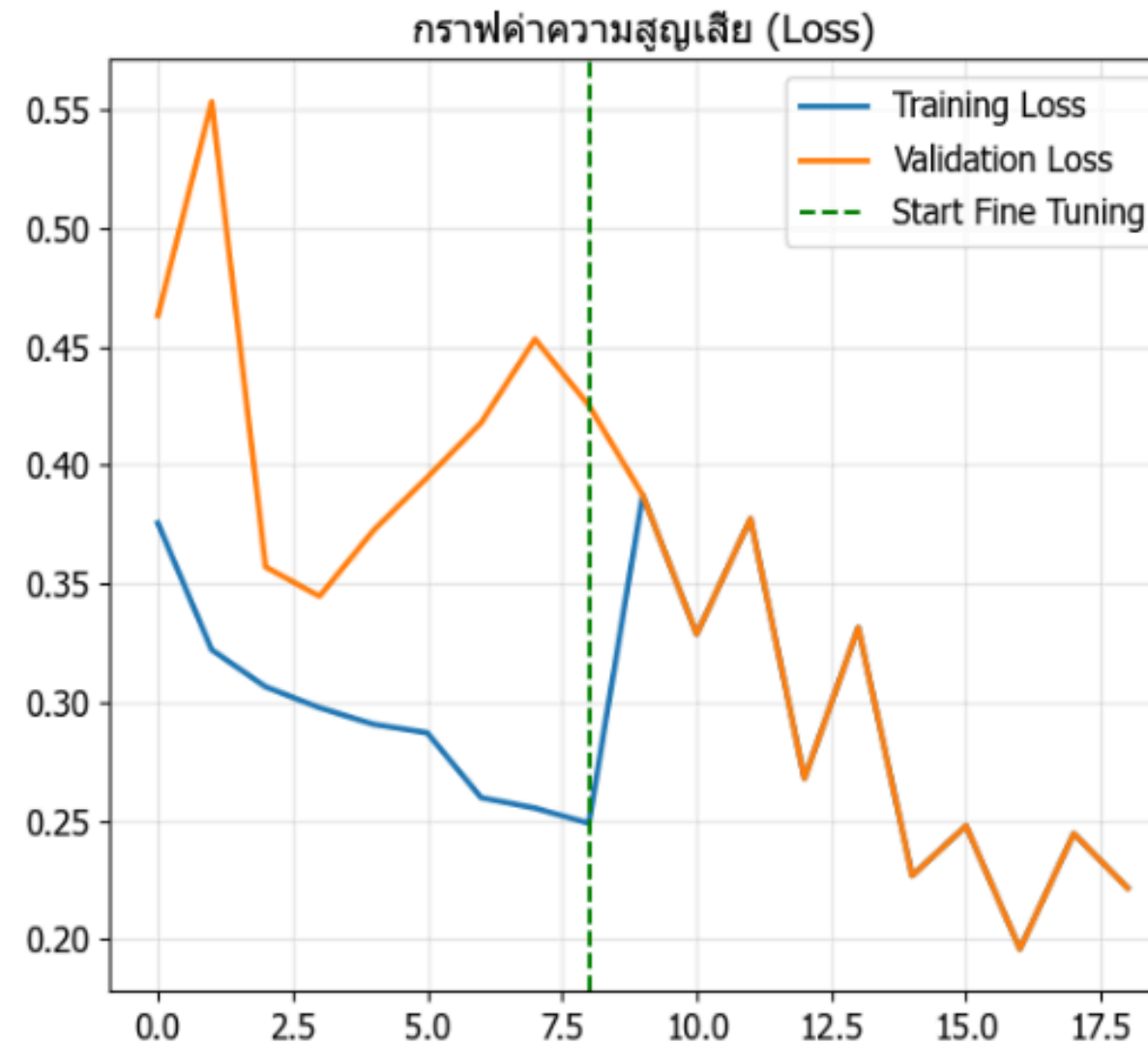


การแสดงผลต่างๆ

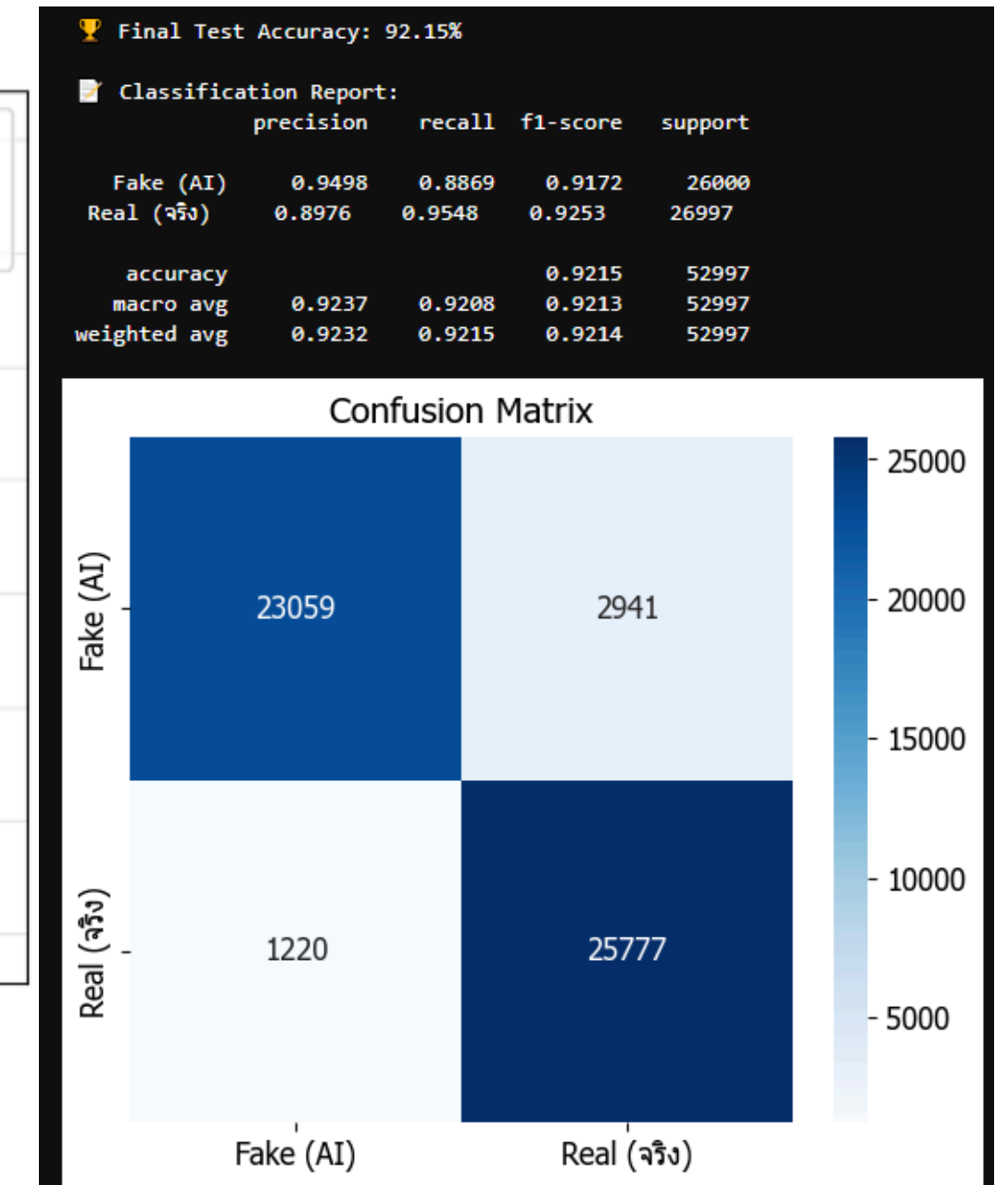
Accuracy



Loss



Confusion Matrix



- High Accuracy โมเดลทำความแม่นยำสูงสุดได้ถึง 92.15% บนชุดข้อมูลทดสอบ (Test Set)
- Fine-Tuning Impact กราฟแสดงให้เห็นชัดเจนว่า เทคนิค Fine-Tuning (หลังเส้นประสีเขียว) เป็นจุดเปลี่ยนสำคัญที่ทำให้ประสิทธิภาพของโมเดลสูงขึ้น
- Low Overfitting ช่องว่างระหว่างเส้นสีน้ำเงินและสีส้มไม่ห่างกันมาก แสดงว่าโมเดลไม่ได้ "จำข้อสอบ" แต่มีความเข้าใจในข้อมูลจริงๆ



SYSTEM OVERVIEW

```
Help web-ai-detect
database.json U README.md Extension: Container Tools fine_tuned_ai_model.h5

frontend > src > App.tsx > App
1 // frontend/src/App.tsx
2 import React, { useState, useRef, useEffect } from "react";
3 import { motion, AnimatePresence } from "framer-motion";
4 import "../App.css";
5 import confetti from "canvas-confetti";
6
7 type Result = {
8   label: string;
9   score: number;
10  confidence: number;
11 };
12
13 export default function App() {
14   const [file, setFile] = useState<File | null>(null);
15   const [preview, setPreview] = useState<string | null>(null);
16   const [result, setResult] = useState<Result | null>(null);
17   const [loading, setLoading] = useState(false);
18   const [progress, setProgress] = useState(0);
19   const apiUrl = import.meta.env.VITE_API_URL || "http://localhost:8000";
20   const containerRef = useRef<HTMLDivElement | null>(null);
21   const progressIntervalRef = useRef<number | null>(null);
22
23   useEffect(() => {
24     return () => {
25       if (preview) {
26         URL.revokeObjectURL(preview);
27       }
28       if (progressIntervalRef.current) {
29         window.clearInterval(progressIntervalRef.current);
30       }
31     };
32     // eslint-disable-next-line react-hooks/exhaustive-deps
33   }, []);
34
35   const onFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
36     const f = e.target.files ? e.target.files[0] : null;
37     if (preview) {
38       try {
39         URL.revokeObjectURL(preview);
40       } catch {
41         // ignore
42       }
43     }
44     setFile(f);
45     setResult(null);
46     setProgress(0);
47     if (f) {
48       const url = URL.createObjectURL(f);
49       setPreview(url);
50       setLoading(true);
51       setProgress(0);
52       progressIntervalRef.current = window.setInterval(() => {
53         setProgress((prev) => prev + 10);
54       }, 100);
55     }
56   };
57 }
```

1. User Interface (Frontend):

พัฒนาด้วย React.js ร่วมกับ Vite เพื่อความรวดเร็วมีการใช้ Interactive UI (Scanner Effect) เพื่อสร้างประสบการณ์การใช้งานที่ทันสมัยรองรับการอัปโหลดไฟล์ภาพ (JPG, PNG, WebP) และแสดงผลลัพท์แบบ Real-time

2. Core AI Engine:

หัวใจหลักคือไฟล์โมเดล fine_tuned_ai_model.h5 (TensorFlow/Keras Format) โมเดลถูกเทรนมาเพื่อจำแนก Binary Classification (Real Image vs AI Generated)

3. Integration Flow (การทำงาน):

Input: ผู้ใช้ส่งรูปภาพผ่านหน้าเว็บ

Preprocessing: ปรับขนาดภาพอัตโนมัติ (เช่น 300x300 px) ให้ตรงกับ Input

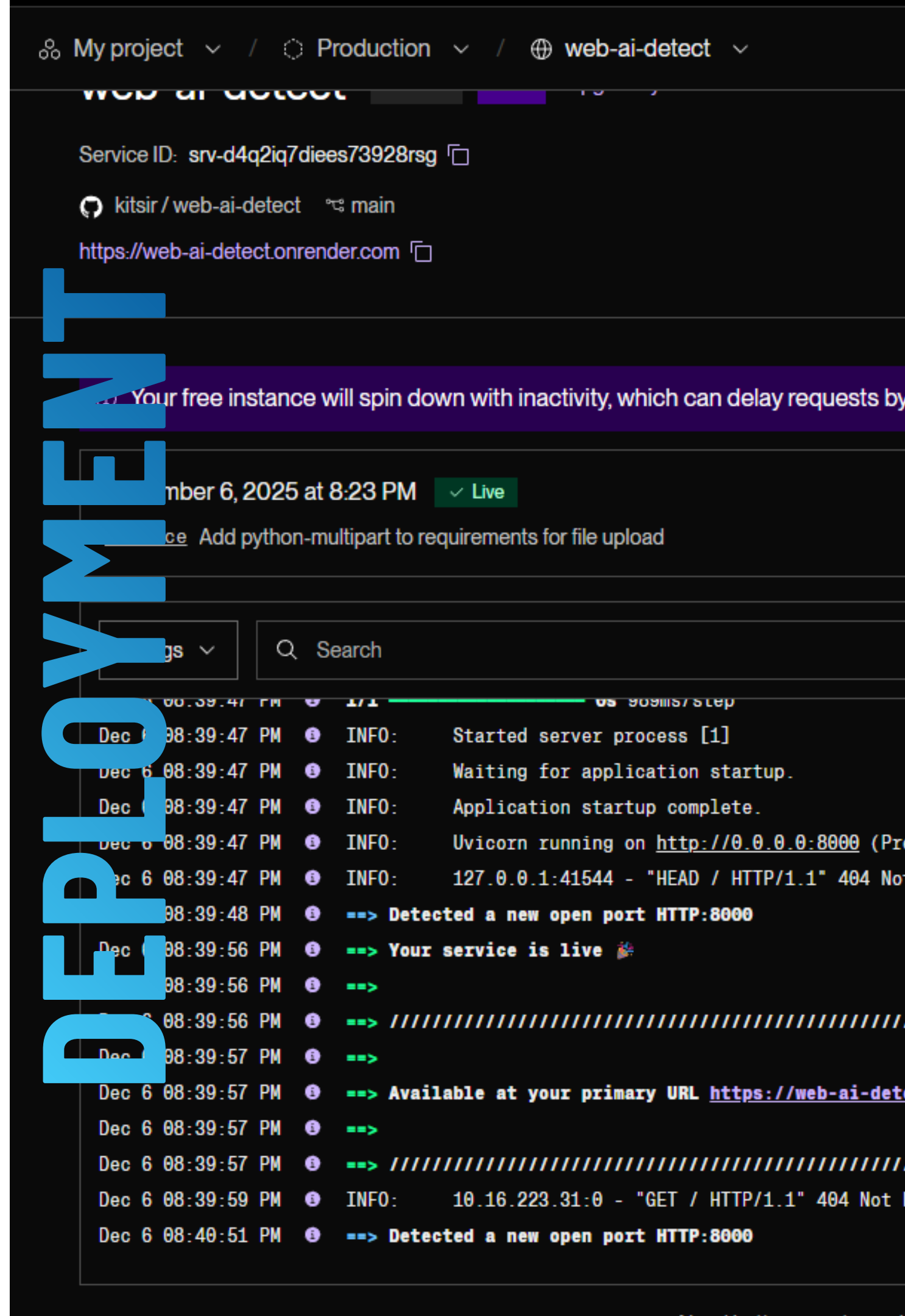
Shape ของโมเดล Inference: โมเดลประมวลผลและให้ค่าความน่าจะเป็น

(Probability Score) Output: หน้าเว็บแปลผลคะแนนเป็นแถบความมั่นใจ

(Confidence Bar) และสถานะ (Real/Fake)



BACKEND ARCHITECTURE & DEPLOYMENT



1. Deployment Environment:

Platform: ใช้งาน Render (Cloud Platform) ในรูปแบบ Web Service
Containerization: ใช้ Docker ในการห่อหุ้ม Environment (Python 3.9 + TensorFlow-CPU) เพื่อให้มั่นใจว่ารันได้ทุกที่เหมือนกัน

2. Model Hosting Strategy:

Storage: ไฟล์โมเดล .h5 ถูกบรรจุ (Embedded) เข้าไปใน Docker Container โดยตรง (ตำแหน่ง /app/backend/)
Reason: เพื่อความเร็วสูงสุดในการเรียกใช้ (Low Latency)
ไม่ต้องเสียเวลาดาวน์โหลดใหม่ทุกครั้งที่ได้รับ

3. API Implementation:

Framework: ใช้ FastAPI (Python) เพราะเร็วและรองรับ Asynchronous
API Endpoint Route: /predict
Method: POST Payload: multipart/form-data (File Upload)
Response Format (JSON):



Meet Our Team



นายกจ วิเศษคามิน
1650705468

มีความคิดอ่านที่กว้างไกลแต่มัก
ใช้ความรู้ในทางที่ไม่ถูกไม่ควร



นายถิรุตม์ ไชยพินิจ
1650707308

เป็นคนที่หาตัวจับได้ยาก
เนื่องจากเรียนเสร็จจะกลับห้อง
เป็นประจำนานๆครั้งจะกินข้าวกับ
เพื่อนๆ



นายทศายุ ปรัชญกุล
1650703745

เป็นคนรักจริงไม่เจ้าชู้ แต่โสด

บอกทำไม